Critique DPO

Zihe Liu, Peter Hayes

March 14, 2026

Abstract

Direct Preference Optimization (DPO) enables fine-tuning language models from pairwise preferences without reinforcement learning, while using binary preference labels, discarding the reasoning behind the oracle's judgement. We propose Critique DPO (CDPO), which augments the DPO loss with an additional negative log-likelihood term on the verifier's chain-of-thought critique, training the base policy to reproduce the step-by-step reasoning that justifies the preference. We evaluate CDPO against DPO and SFT baselines on Qwen2.5-0.5B-Instruct using 120,000 math preference triples from the GenRM dataset. Our results are a negative finding: both DPO and CDPO suffer performance collapse on GSM8K and MATH-500, with CDPO collapsing faster and more severely than vanilla DPO. We analyse this failure through the lens of reward hacking, distributional mismatch, and discuss implications for when and how verifier critique can be incorporated into base policy training.

1 Introduction

As large language models (LLMs) become more capable, fine-tuning techniques to align with human intent are increasingly important. A key consideration for aligning these models is how to most effectively use human resources, or model resources in the case where LLMs themselves are used as oracles. Reinforcement learning from Human or AI preferences (RLHF/RLAIF) is the most prominent example of such a technique but is complex and often unstable. Direct Preference Optimization (DPO) [Rafailov et al., 2024] has recently been proposed as a simpler and more stable alternative, bypassing the reward modelling and RL steps entirely. However, DPO reduces the oracle's judgement to a single binary label — preferred or rejected — discarding any reasoning the oracle provides for its preference, using only the COT of the base model for DPO training.

Asking these oracles to provide reasoning for their binary preferences is common practice. In particular, for LLM-as-a-judge style workflows, asking the model to provide chain-of-thought reasoning has been shown to improve the quality of the judgements themselves. This raises a natural question: can we leverage the oracle's reasoning — not just its binary label — when fine-tuning the target model?

In this work, we propose Critique DPO (CDPO), which augments the DPO objective with a negative log-likelihood term on the verifier's chain-of-thought critique, training the base policy to reproduce the step-by-step verification reasoning alongside learning from pairwise preferences. We evaluate CDPO on mathematical reasoning using Qwen2.5-0.5B-Instruct and the GenRM dataset. Our results are a negative finding: CDPO collapses faster and more severely than vanilla DPO, which itself suffers catastrophic performance degradation at this smaller scale. We analyse this failure and discuss implications for the design of critique-augmented training methods.

2 Background: DPO

For a given prompt x, two completions are sampled from the policy $(y_w, y_l) \sim \pi_\theta(y|x)$ and an oracle judges which it prefers. We denote y_w as the preferred completion and y_l as the other. This process is repeated over N prompts resulting in the pairwise preference dataset $\mathcal{D} = \{x^{(i)}, y_w^{(i)}, y_l^{(i)}\}_{i=1}^N$.

2.1 Bradley-Terry Preference Model

We can model the judge preferences using the Bradley-Terry (BT) model, which expresses the probability that completion y_1 is preferred over y_2 under a latent reward function r^*:

$$p^*(y_1 \succ y_2 \mid x) = \frac{\exp\left(r^*(x, y_1)\right)}{\exp\left(r^*(x, y_1)\right) + \exp\left(r^*(x, y_2)\right)} = \sigma(r^*(x, y_1) - r^*(x, y_2))$$

A key property is that BT depends only on the *difference* of rewards, not their absolute values.

2.2 Standard RLHF

The standard RLHF pipeline comprises 2 stages. First, we learn a reward model r_ϕ where we find the maximum likelihood estimate (MLE) of ϕ under our preference dataset $\mathcal{D}$ and BT model - termed reward modelling. Then, we use RL (PPO) to learn an optimal policy π_θ for this learned reward model.

2.2.1 Reward Modelling

Since r^* is unknown, we learn a parametric approximation r_ϕ. Under the BT model, the likelihood of the observed preference dataset is $\ell(\mathcal{D} \mid \phi) = \prod_{i=1}^{N} \sigma\Big(r_\phi(x^{(i)}, y_w^{(i)}) - r_\phi(x^{(i)}, y_l^{(i)})\Big)$. Taking the log and writing as an expectation, the MLE is obtained by minimising

$$\mathcal{L}_{\text{RM}}(\phi) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))\right] \tag{1}$$

2.2.2 RL Fine-Tuning

The learned reward model is then used to fine-tune the policy. The aim is to maximise the KL-constrained objective

$$\max_{\pi_\theta} \; \mathbb{E}_{x\sim\mathcal{D},\, y\sim\pi_\theta(y|x)}\left[r_\phi(x, y)\right] - \beta\, D_{\text{KL}}(\pi_\theta(y|x) \| \pi_{\text{ref}}(y|x)) \tag{2}$$

where π_{ref} is the reference policy (typically the SFT model), kept frozen. The KL term prevents the policy from deviating too far from the distribution on which the reward model is accurate, and $\beta > 0$ controls this trade-off. In practice this is optimised with PPO, requiring four models in memory: the policy, reference, reward model, and value function.

2.3 DPO

DPO [Rafailov et al., 2024] bypasses the reward modelling and RL steps entirely through a change of variables. The key insight is that there exists a bijection between reward functions and their corresponding optimal policies under the KL-constrained objective (2), and this bijection can be exploited to reparameterise the preference likelihood directly in terms of the policy. We follow the expositions by [Wolfe, 2025] and [Romero, 2024] in our presentation of the derivation below.

2.3.1 Optimal Policy

For *any* reward function r (not just the learned r_ϕ or the latent r^*), the KL-constrained objective (2) can be rewritten as a minimisation of a KL divergence between π and a target distribution defined by r, plus a constant. Specifically, letting $Z_r(x) = \sum_y \pi_{\text{ref}}(y|x) \exp\Big(\frac{1}{\beta} r(x, y)\Big)$ be a partition function that depends on x and r but is constant with respect to π, the objective becomes

$$\min_{\pi} \; \mathbb{E}_x\left[D_{\text{KL}}\left(\pi(y|x) \;\Big\|\; \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\Big(\tfrac{1}{\beta}\, r(x, y)\Big)\right) - \log Z_r(x)\right]$$

By Gibbs' inequality ($D_{\text{KL}} \geq 0$, with equality iff the distributions are identical), the unique optimum is

$$\pi_r(y|x) = \frac{1}{Z_r(x)}\, \pi_{\text{ref}}(y|x)\, \exp\left(\frac{1}{\beta}\, r(x, y)\right) \tag{3}$$

This defines a bijection: every reward r maps to a unique optimal policy π_r, and conversely every policy π_r defines a unique reward[1] r for which it is optimal. Using the latent reward r^* gives the optimal policy $\pi^* \equiv \pi_{r^*}$ for our latent rewards.

2.3.2 Reparameterisation

Rearranging (3) for the reward expresses any reward purely in terms of its corresponding optimal policy and a prompt-dependent partition function:

$$r(x, y) = \beta \log \frac{\pi_r(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_r(x)$$

Substituting the latent reward $r^*(x, y) = \beta \log \frac{\pi^*(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z_{r^*}(x)$ into the BT model, the $\beta \log Z_{r^*}(x)$ terms cancel because BT depends only on reward *differences* and $Z_{r^*}(x)$ is independent of y:

$$p^*(y_w \succ y_l \mid x) = \sigma\left(\beta \log \frac{\pi^*(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi^*(y_l|x)}{\pi_{\text{ref}}(y_l|x)}\right)$$

The preference model now has no dependence on a reward model — it is expressed entirely in terms of policies.

2.3.3 DPO Loss

We replace the fixed π^* with a learnable π_θ and maximise the likelihood of the observed preferences, conditioned on our policy parameters θ. Defining the *implicit reward*

$$\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)} \tag{4}$$

the DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)\right)\right] \tag{5}$$

This is identical in form to the reward model loss (1), but conditioned on the policy parameters θ instead of the reward model parameters ϕ. Traditionally, RLHF performs MLE on $\mathcal{D}$ conditioned on ϕ to learn a reward model, then uses RL to find the optimal policy for that reward. DPO instead performs MLE on $\mathcal{D}$ conditioned directly on θ. Because of the reward–policy bijection, improving π_θ to better fit the preference data simultaneously yields an implicit reward $\hat{r}_\theta$ for which π_θ is guaranteed to be optimal. This eliminates both the reward modelling step and the RL step, requiring only two models in memory (the policy π_θ and the frozen reference π_{ref}).

In this work we suggest simple extensions to this objective.

3 Literature Review

We review research incorporating reasoning and critique data into fine-tuning, building toward the critique-augmented preference optimisation approach of this work. We adopt a unified notation: π_θ denotes the learnable policy, π_{ref} the frozen reference (usually the base SFT model), x the prompt, y_w and y_l the preferred and rejected responses, c a model's own chain-of-thought reasoning trace, c_v a verifier's chain-of-thought for the critique, whether from an external oracle or generated by the model itself, $r_\phi(x, y)$ a learned scalar reward, and $\hat{r}_\theta(x, y) = \beta \log \frac{\pi_\theta(y|x)}{\pi_{\text{ref}}(y|x)}$ the implicit reward.

The review is organised around a central question: *what kind of signal is being used, and for which model?* The narrative moves from binary AI feedback (Section 3.1), through enriching DPO with supervised reasoning targets (Section 3.2), to methods that improve verifiers by having them reason (Section 3.3), and finally to the closest work — methods that bring verifier critique directly into the base model's training objective (Section 3.4).

[1] We really mean unique equivalence class of rewards and its optimal policy; see Lemma 1 and Lemma 2 in [Rafailov et al., 2024].

3.1 Beyond Binary Feedback

3.1.1 RLAIF and the Information Bottleneck

RL from AI Feedback (RLAIF) replaces human annotators with LLM oracles. [Bai et al., 2022] first use LLM-generated labels for harmlessness finetuning, doing RL with a preference model trained with LLM preferences. [Lee et al., 2024] use LLM to directly give rewards, showing that RLAIF matches or exceeds RLHF on summarisation and dialogue, even when the labeller is the same model being aligned. [Li et al., 2024] instruct the labeller to reason explicitly and report modest improvements. A shared limitation is that the oracle's reasoning is discarded after labelling: the sole training signal is a binary or scalar preference label, creating an information bottleneck that the remainder of this review seeks to address.

3.1.2 Impact of Chain-of-Thought Data for DPO

[H. Liu et al., 2025] provide a systematic empirical study of CoT's impact on DPO. In text-to-SQL — where preference datasets traditionally contain only final SQL queries *without* reasoning steps — DPO degrades performance across the majority of ten tested LLMs. Augmenting the same datasets with synthetically generated chain-of-thought solutions achieves consistent DPO improvements for the first time. The analysis identifies three mechanisms. First, *reward hacking mitigation*: without CoT, the implicit reward (4) can be inflated by superficial correlates (longer SQL, non-executable patterns) that do not reflect reasoning quality. Model performance initially peaks then decreases, while the implicit reward continues to rise. With CoT, the model's self-assessed implicit reward tracks actual performance. Second, *discriminative calibration*: CoT-trained models maintain stable classification accuracy between correct and incorrect solutions throughout training, whereas vanilla DPO becomes unstable. Third, *sample efficiency*: CoT-augmented DPO continues to improve with additional data, while vanilla DPO saturates. 2 samples of CoT with DPO achieves the same performance gains as 16 samples of vanilla DPO.

This finding directly motivates our approach: if the model's own CoT improves DPO, then an external verifier's critique — which provides a richer perspective on why one response is preferred — should be informative too.

3.1.3 Self-Taught Reasoning: STaR and V-STaR

STaR. [Zelikman et al., 2022] establish that CoT data, even when self-generated, is a richer training signal than outcome labels alone. Given a pretrained model M and a dataset $\mathcal{D} = \{(x_i, y_i)\}$ of problems with ground-truth answers, STaR begins with a small few-shot prompt set $\mathcal{P}$ containing rationale examples. At each iteration, the model generates a reasoning trace $\hat{c}_i$ and answer $\hat{y}_i$ for every problem, using $\mathcal{P}$ in the context. Only correct generations where $\hat{y}_i = y_i$ are retained; the model is then fine-tuned on this filtered set with the standard NLL objective on rationale-answer pairs:

$$\mathcal{L}_{\text{STaR}}(M) = -\mathbb{E}_{(x,y)\sim\mathcal{D}}\left[\mathbf{1}[\hat{y} = y] \cdot \sum_t \log M(\hat{c}_t, \hat{y}_t \mid x, \hat{c}_{<t})\right] \tag{6}$$

STaR with rationalization matches GPT-3 (30× larger) on CommonsenseQA while using only 86.7% of training data. The method demonstrates that self-generated rationales – even produced with $|\mathcal{P}| = 10$ few-shot examples – provide a dramatically richer training signal than training on final answers alone.

V-STaR. [Hosseini et al., 2024] identify the core inefficiency of STaR: the large number of *incorrect* self-generated solutions is discarded, despite containing information about error patterns. V-STaR repurposes these failures to co-train a generator G and a verifier V in parallel across T iterations.

They maintain two accumulating datasets: $\mathcal{D}_{\text{GEN}}$: correct solutions $(x, \hat{y})$, used for generator SFT, and $\mathcal{D}_{\text{VER}}$: *all* generated solutions $(x, \hat{y}, z)$, used for verifier training. At each iteration t, $k = 16$ solutions per problem are sampled from G^t and automatically labeled for correctness — by comparing the extracted final answer against the ground-truth answer for math, or by running test cases for code. Correct solutions are appended to $\mathcal{D}_{\text{GEN}}$; all solutions are appended to $\mathcal{D}_{\text{VER}}$. G^t is fine-tuned on $\mathcal{D}_{\text{GEN}}^{<t}$ with the standard NLL loss Eq. (6). From $\mathcal{D}_{\text{VER}}$, preference pairs (x, y^+, y^-) are formed by taking the pair of all correct solutions y^+ and all incorrect solutions y^- for each problem x. The verifier is trained with DPO

using G_{SFT} (the base model fine-tuned once) as the fixed reference policy:

$$\mathcal{L}_{\text{DPO}}(V^t;\, G_{\text{SFT}}) = -\mathbb{E}_{(x,y^+,y^-)\sim\mathcal{D}_{\text{VER}}}\left[\log\sigma\big(\hat{r}(x,y^+) - \hat{r}(x,y^-)\big)\right], \quad \hat{r}(x,y) = \beta\log\frac{V^t(y \mid x)}{G_{\text{SFT}}(y \mid x)} \tag{7}$$

At inference time, the verifier is used in Best-of-k reranking to choose solutions for better performance. Crucially, the verifier provides no training-time signal to the generator. The generator is trained purely via SFT on correct solutions (identical to STaR). The verifier's direct contribution is better performance at inference via Best-of-k reranking. As the generator improves across iterations it produces harder-to-distinguish incorrect solutions, enriching $\mathcal{D}_{\text{VER}}$. However this does not feed any gradient back into the generator's training. This distinguishes V-STaR from later methods. Nevertheless, V-STaR 7B surpasses LLaMA-2 70B (8-shot) on GSM8K and nearly matches CodeLLaMA 34B on HumanEval in out-of-domain transfer, which illustrates the value of CoT training signals.

3.1.4 Self-Rewarding Language Models

[Yuan et al., 2024] propose having the model act as its own reward model: the *same* model M_t both generates candidate responses and scores them. To bootstrap this self-evaluation capability, the model is first SFT'd on seed evaluation data (EFT) — examples of (instruction, response, CoT justification, score) tuples that teach it to write a reasoning trace followed by a scalar score out of 5 using an additive rubric.

At each iteration t, M_t: (1) generates $N = 4$ candidate responses per prompt x; (2) re-prompts *itself* with a judge template to score each candidate, producing a scalar $r_t(x, y) \in \{0, \ldots, 5\}$ averaged over 3 independent samples to reduce variance; (3) forms preference pair (y_w, y_l) from the highest- and lowest-scoring responses (ties discarded). The preference pairs are used to train M_{t+1} via DPO ($\beta = 0.1$).

The reward model is *not frozen* — it co-evolves with the policy, and the quality of the preference data improves across iterations. Pairwise reward accuracy rises from 65.1% (SFT baseline) to 81.7% (M3). On AlpacaEval 2.0, win rates improve from 9.94% (M1) → 15.38% (M2) → 20.44% (M3), with M3 outperforming Claude 2, Gemini Pro, and GPT-4 0613 despite using only 3,200 seed examples.

3.2 DPO with NLL

A recurring structural pattern in the literature combines the DPO preference loss with an auxiliary negative log-likelihood (NLL) term on a supervised target. We present the generic template first, then show how each paper instantiates it with a **different NLL target** t:

$$\mathcal{L}_{\text{DPO+NLL}} = \underbrace{-\log\sigma(\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l))}_{\text{preference term}} + \alpha\ \underbrace{\mathcal{L}_{\text{NLL}}(t \mid x)}_{\text{supervised anchor}} \tag{8}$$

The DPO term provides a contrastive signal (push apart y_w and y_l), while the NLL term anchors the policy to a specific supervised target. As we show below, the choice of t is what distinguishes these methods from one another.

3.2.1 Iterative Reasoning Preference Optimization (IRPO)

[Pang et al., 2024] set the NLL target to the *model's own winning chain-of-thought*: $t = (c_w, y_w)$. At each iteration, the model samples N reasoning traces per prompt, partitions them into correct (c_w, y_w) and incorrect (c_l, y_l) by checking the final answer against ground truth, and optimises:

$$\begin{aligned}\mathcal{L}_{\text{IRPO}} &= \mathcal{L}_{\text{DPO}}(c_w, y_w, c_l, y_l \mid x) + \alpha\,\mathcal{L}_{\text{NLL}}(c_w, y_w \mid x) \\ &= -\log\sigma\left(\beta\log\frac{\pi_\theta(c_w, y_w \mid x)}{\pi_{\text{ref}}(c_w, y_w \mid x)} - \beta\log\frac{\pi_\theta(c_l, y_l \mid x)}{\pi_{\text{ref}}(c_l, y_l \mid x)}\right) - \alpha\frac{\log\pi_\theta(c_w, y_w \mid x)}{|c_w| + |y_w|}\end{aligned} \tag{9}$$

The NLL term is length-normalised by $|c_w| + |y_w|$ to avoid a bias toward shorter sequences. Crucially, the reference model π_{ref} is updated to π_θ at each iteration before re-sampling. Under standard DPO (i.e. $\alpha = 0$), the log-probabilities of *chosen* sequences actually decrease during training — the model achieves the required preference margin by pushing rejected probabilities down faster than chosen ones fall. The NLL anchor prevents this collapse: with $\alpha > 0$, chosen log-probabilities increase while rejected ones decrease, creating a healthy expanding margin. Empirically, DPO alone achieves 61.8% on GSM8K while DPO+NLL achieves 73.1% at iteration 1; after four iterations.

3.2.2 AIPO: Agreement-Aware Iterative Preference Optimization

[Shen et al., 2024] set the NLL target to the **winning response**: $t = y_w$. Their key modification is to the DPO term itself. Defining the log-probability ratios

$$s_\theta = \log \frac{\pi_\theta(y_w \mid x)}{\pi_\theta(y_l \mid x)}, \qquad s_{\text{ref}} = \log \frac{\pi_{\text{ref}}(y_w \mid x)}{\pi_{\text{ref}}(y_l \mid x)},$$

standard DPO is $-\log \sigma(\beta(s_\theta - s_{\text{ref}}))$. AIPO introduces an agreement-aware coefficient $\alpha > 0$:

$$\mathcal{L}_{\alpha\text{-DPO}} = -\mathbb{E}_{\mathcal{D}}\big[\log \sigma\big(\beta\big(s_\theta - (1+\alpha)\, s_{\text{ref}}\big)\big)\big] \tag{10}$$

The gradient weight decomposes as $\sigma(\beta(s_{\text{ref}} - s_\theta + \alpha\, s_{\text{ref}}))$: the additional $\alpha\, s_{\text{ref}}$ amplifies the learning signal when the reference model agrees with the preference label and dampens it when there is disagreement, mitigating the length exploitation that compounds across iterative DPO. The full AIPO loss combines this with a length-normalised NLL term:

$$\mathcal{L}_{\text{AIPO}} = \mathcal{L}_{\alpha\text{-DPO}} + \lambda \cdot \mathcal{L}_{\text{NLL}}, \qquad \mathcal{L}_{\text{NLL}} = -\frac{1}{|y_w|} \log \pi_\theta(y_w \mid x) \tag{11}$$

AIPO achieves 57.1% length-controlled win rate on AlpacaEval 2.0 with Gemma-2-9B-It and 81.6% on Arena-Hard with Mistral-Large, outperforming GPT-4.

3.2.3 Regularized Preference Optimization (RPO)

[Z. Liu et al., 2024] set the NLL target to the **chosen responses**: $t = y_w$ — the same target as AIPO — but derive this choice from first principles rather than empirical motivation. They frame DPO overoptimization as a distributional shift problem and seek a policy that performs well against an adversarially chosen reward model: one that simultaneously minimises the BT preference loss and a penalty discouraging the policy from high-reward out-of-distribution responses:

$$\hat{\pi} \in \arg\max_{\pi} \min_{r} \{\mathbb{E}_{x,\, a\sim\pi}[r(x,a)] - \beta\, D_{\text{KL}}(\pi \| \pi_{\text{ref}}) + \eta\, \mathcal{L}_{\mathcal{D}}(r)\} \tag{12}$$

Applying the DPO reparametrisation (3) and solving the inner maximisation in closed form, the minimax objective collapses exactly to DPO plus NLL on chosen responses:

$$\mathcal{L}_{\text{RPO}}(\theta) = \mathcal{L}_{\text{DPO}}(\theta) \;-\; \eta\beta\, \mathbb{E}_{(x,y_w)\sim\mathcal{D}}[\log \pi_\theta(y_w \mid x)] \tag{13}$$

The key theoretical contribution is that the NLL term is the exact residue of the adversarial penalty after reparametrisation, making DPO+NLL on chosen responses a provably sample-efficient solution under partial data coverage. Since DPO already computes $\log \pi_\theta(y_w|x)$ in its forward pass, RPO adds zero compute overhead. RPO outperforms DPO on MT-Bench (7.38 vs. 7.28 on Mistral-7B) and AlpacaEval 2.0 LC win rate (23.3% vs. 21.2%).

3.2.4 Direct Judgement Preference Optimization (DJPO)

[P. Wang et al., 2024] set the NLL target to the **verifier's CoT critique and judgment**: $t = (c_v, j)$, where c_v is the verifier's chain-of-thought critique and j is the final judgment. Crucially, the model being trained is a *judge*, not the base policy. They construct three types of DPO preference pairs — CoT critique $\mathcal{D}_{\text{CoT}}$, standard judgment $\mathcal{D}_{\text{Std}}$, and response deduction $\mathcal{D}_{\text{Ded}}$ — and train the collective preference dataset with:

$$\mathcal{L}_{\text{DJPO}} = -\frac{\log \pi_\theta(y_w \mid x)}{|y_w| + |x|} - \log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right) \tag{14}$$

where y_w and y_l are correct and incorrect judge evaluations respectively. Using 680K preference pairs with a 70:15:15 ratio across the three types, they train SFR-Judge models at 8B, 12B, and 70B scales. SFR-Judge-8B outperforms GPT-4o on aggregate pairwise evaluation across 7 benchmarks; SFR-Judge-70B achieves the best overall score of 84.25%.

3.3 Generative Verifiers and Critique-Augmented Reward Models

The methods above train the policy model. A parallel line of work improves the verifiers and reward models that judge responses, by having them produce reasoning before scoring. These methods establish that critique data is a powerful training signal for judging. Generative verifiers aim to provide a scalar verifier score that is used during inference for Best-of-N sampling, whereas critique-augmented reward models produce better reward models for downstream preference tuning etc.

3.3.1 Generative Verifiers (GenRM)

[Zhang et al., 2024] observe that discriminative reward models require a separate classification head to assign scalar rewards and cannot use the LLM's generative capabilities — they cannot reason step-by-step before scoring, and they cannot leverage additional inference-time compute. Generative verifiers (GenRM) address this by recasting verification as next-token prediction, training the verifier with the standard SFT loss. Gemini 1.0 Pro generates solutions for GSM8K while a Gemma model is fine-tuned as the verifier.

In the simplest variant (**GenRM-Direct**), given instruction I = "Is the answer correct (Yes/No)?", the verifier is trained on:

$$\mathcal{D}_{\text{Direct}} = \{(x, y^+, I),\ \text{'Yes'}\} \cup \{(x, y^-, I),\ \text{'No'}\} \tag{15}$$

where y^+ and y^- are correct and incorrect solutions labelled by an external oracle. At inference, the verifier score is the next-token probability of 'Yes':

$$r_{\text{Direct}}(x, y) = \pi_\theta(\text{'Yes'} \mid x, y, I) \tag{16}$$

which is used with Best-of-N sampling to choose the best generator solution at inference time.

Unifying generation and verification. Training on correct solution generation provides positive transfer to verification. The default training objective for the verifier mixes both tasks:

$$\mathcal{L}_{\text{GenRM}}(\theta) = \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{verify}}) + \lambda\, \mathcal{L}_{\text{SFT}}(\theta, \mathcal{D}_{\text{correct}}) \tag{17}$$

where $\mathcal{D}_{\text{verify}}$ is either $\mathcal{D}_{\text{Direct}}$ or $\mathcal{D}_{\text{CoT}}$, and $\mathcal{D}_{\text{correct}}$ is a set of correct solutions from the training set.

GenRM-CoT. Since verification requires nuanced reasoning, GenRM-CoT inserts a chain-of-thought rationale c_v before the final Yes/No token. The dataset is:

$$\mathcal{D}_{\text{CoT}} = \big\{(x, y^+, I_v),\ (c_v, I, \text{'Yes'})\big\} \cup \big\{(x, y^-, I_v),\ (c_v, I, \text{'No'})\big\}$$

where I_v = "Let's verify step by step". The rationales c_v are generated offline by a capable LLM. Rationales are filtered by majority agreement across multiple generated samples. The verifier itself generates no c_v during training; only at inference does it sample its own rationale and then score:

$$r_{\text{CoT}}(x, y) = \pi_\theta(\text{'Yes'} \mid x, y, I_v, c_v, I), \qquad c_v \sim \pi_\theta(\cdot \mid x, y, I_v) \tag{18}$$

Multiple independent rationales can be sampled and aggregated via majority voting to use additional test-time compute:

$$r_{\text{MV@}K}(x, y) = \frac{1}{K} \sum_{i=1}^{K} \pi_\theta\Big(\text{'Yes'} \mid x, y, I_v, c_v^{(i)}, I\Big), \quad c_v^{(i)} \overset{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid x, y, I_v) \tag{19}$$

The whole procedure is a single-pass offline SFT; the trained verifier is used only at inference for Best-of-N reranking. GenRM-CoT achieves 94.2% Best-of-32 on GSM8K (vs. ∼90% for a discriminative RM), and generalises to MATH500 trained only on GSM8K with 6.4× better sample efficiency than the discriminative baseline. GenRM also outperforms DPO-trained verifiers substantially, with the best DPO scoring strategy (raw log-likelihood, without subtracting the reference) still underperforming GenRM-Direct.

3.3.2 GenRM-STaR-DPO: Training Judges with DPO on Rationales

[Mahan et al., 2024] (Mahan et al., 2024) propose a unified RLHF–RLAIF framework built on a key theoretical observation: under the Bradley-Terry model, the standard RLHF reward-difference objective is equivalent to a preference log-odds objective,

$$r_\phi(x, y) - r_\phi(x, y_{\text{ref}}) = \log \frac{p_{\text{BT}}(y \succ y_{\text{ref}} \mid x)}{1 - p_{\text{BT}}(y \succ y_{\text{ref}} \mid x)} \tag{20}$$

This means one can replace the scalar reward parameterisation with a fully general preference model $p_\phi(y_w \succ y_l \mid x)$ extracted directly from an LLM's token probabilities, without requiring a separate classification head or a specific distributional assumption. This is the conceptual foundation for training a generative judge as a reward model.

Three training variants. Given a preference dataset $\mathcal{D} = \{(x, y_1, y_2, I)\}$ where $I \in \{A, B\}$ is the winning response indicator:

GenRM (no CoT) — standard SFT on the indicator token:

$$\mathcal{L}_{\text{GenRM}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,I)\sim\mathcal{D}}[-\log \pi_\phi(I \mid x, y_1, y_2)] \tag{21}$$

CoT-GenRM with Rationalization — a post-rationalization model q (e.g. GPT-4 or a stronger LLM) generates a critique rationale c_v conditioned on the correct answer, and the model is trained on both:

$$\mathcal{L}_{\text{Rationalizer}}(\pi_\phi) = \mathbb{E}_{(x,y_1,y_2,c_v,I)\sim\mathcal{D}}[-\log \pi_\phi(I \mid x, y_1, y_2, c_v) - \log \pi_\phi(c_v \mid x, y_1, y_2)] \tag{22}$$

CoT-GenRM-STaR-DPO — instead of discarding incorrect self-generated critique rationales (as STaR-SFT does), STaR-DPO constructs preference pairs $(c_{v,w}, I_w)$ vs $(c_{v,l}, I_l)$ from correct and incorrect reasoning traces and applies DPO over the full rationale-plus-judgment sequence:

$$\mathcal{L}_{\text{STaR-DPO}}(\pi_\phi) = -\mathbb{E}_{\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\phi(I_w, c_{v,w} \mid x, y_1, y_2)}{\pi_{\text{ref}}(I_w, c_{v,w} \mid x, y_1, y_2)} - \beta \log \frac{\pi_\phi(I_l, c_{v,l} \mid x, y_1, y_2)}{\pi_{\text{ref}}(I_l, c_{v,l} \mid x, y_1, y_2)}\right)\right] \tag{23}$$

The model learns not just which response is correct but which *reasoning patterns* lead to correct judgments, and this contrastive signal is what drives OOD generalisation.

On-policy rationales are essential: using rationales from a strong off-policy model (GPT-4 or Llama-3.1-70B) as the training signal — the STaR-Rationalizer approach — matches STaR-DPO in-distribution but collapses OOD (RewardBench accuracy drops to 63–70%, below even the zero-shot LLM-as-a-Judge). The authors attribute this to distribution mismatch: the model learns to imitate the rationalizer's style in-distribution but generates different reasoning at test time. STaR-DPO avoids this by training on the model's own self-generated rationales throughout. STaR-DPO matches the best explicit reward models in-distribution (73.9% vs 73-74%) while outperforming all of them OOD, particularly on Safety (91.0% vs 81.8% for PairRM). On reasoning-specific tasks (UltraInteract training, RewardBench Reasoning), STaR-DPO achieves 87.2% vs 70.8% for the best non-reasoning model.

3.3.3 Critique-out-Loud Reward Models (CLoud)

[Ankner et al., 2024] train reward models that generate a natural-language critique c before predicting a scalar reward. The model has 2 parameter groups built on top of a shared base θ_B: θ_{LM}: the pretrained LM head, which generates critique tokens autoregressively, and θ_R: a randomly initialised shallow MLP reward head, which reads the final hidden state of the full sequence and outputs a scalar reward. Scoring at inference is therefore a two-step procedure: (1) *generate* a critique by feeding (x, y) through $\theta_B + \theta_{LM}$ to autoregressively sample c; then (2) *score* by feeding the full sequence (x, y, c) through $\theta_B + \theta_R$ to obtain the scalar $r_{\theta_R}(x, y, c)$. The training pipeline is as follows:

Stage 1 — SFT on oracle critiques External oracle critiques are generated offline by Llama-3.1-405B for every chosen and rejected response. θ_B, θ_{LM} is fine-tuned on these via NLL; (θ_R does not yet exist):

$$\mathcal{L}_{\text{SFT}}(\theta_B, \theta_{LM}) = -\mathbb{E}\left[\sum_t \log p(c_t^- \mid x, y^-, c_{<t}^-) + \sum_t \log p(c_t^+ \mid x, y^+, c_{<t}^+)\right] \tag{24}$$

Stage 2 — Self-generated Critique Using the Stage 1 model, new critiques are sampled for every training example: $\hat{c}^{\pm} \sim p(\cdot \mid x, y^{\pm}; \tilde{\theta}_B, \tilde{\theta}_{LM})$. These replace the oracle critiques to form $\mathcal{D}_{\text{self}}$. At inference θ_R will see self-generated critiques, so we need to produce good self-generated critiques to reduce distributional shift.

Stage 3 — Joint training on $\mathcal{D}_{\text{self}}$. θ_R is randomly initialised on top of θ_B . All three parameter groups are jointly optimised on $\mathcal{D}_{\text{self}}$:

$$\mathcal{L}_{\text{CLoud}} = \underbrace{-\mathbb{E}\big[\log \sigma\big(r_{\theta_R}(x, y^+, \hat{c}^+) - r_{\theta_R}(x, y^-, \hat{c}^-)\big)\big]}_{\mathcal{L}_{\text{RM}}\text{: trains } \theta_R \text{ and } \theta_B} + \lambda \underbrace{\mathcal{L}_{\text{SFT}}(\theta_B, \theta_{LM}, \mathcal{D}_{\text{self}})}_{\text{trains } \theta_{LM} \text{ and } \theta_B} \tag{25}$$

$\mathcal{L}_{\text{RM}}$ sends gradients into θ_R via the classic Bradley-Terry preference modelling loss; $\mathcal{L}_{\text{SFT}}$ sends gradients into θ_{LM}. The $\mathcal{L}_{\text{SFT}}$ term in Stage 3 is important — without it, θ_{LM} receives no gradient and critique quality degrades, which then corrupts the conditioning signal seen by θ_R.

At inference, N critiques are sampled and the resulting scalar rewards averaged:

$$\bar{R}(x, y) = \frac{1}{N} \sum_{i=1}^{N} r_{\theta_R}(x, y, c_i)\,, \qquad c_i \sim p(\cdot \mid x, y;\, \theta_B, \theta_{LM}) \tag{26}$$

On-policy critiques are essential: training the reward head on oracle (off-policy) critiques causes a 5.6% drop in RewardBench accuracy for the 8B model (3.0 pp for 70B) relative to self-generated (on-policy) training. The reward head learns to condition on the statistical properties of the critique distribution it saw during training; oracle critiques are systematically longer and more detailed than self-generated ones, so the reward head cannot properly calibrate from a distribution shift critique at inference. CLoud-8B improves pairwise RewardBench accuracy by 4.65% over a classic RM and is Pareto-dominant across all $N \in [2, 16]$ on ArenaHard Best-of-N win rate. The 8B CLoud model outperforms the 70B classic RM on Chat and Safety categories.

3.3.4 Critic-RM: Critiques as Latent Variables

[Yu et al., 2025] provide a principled variational formulation that unifies critique generation and reward modeling, requiring no teacher LLM — critiques are self-generated and quality-filtered.

The model is built on a shared backbone M_θ with two heads: $q_\phi = h_g \circ M_\theta$ (the pretrained LM head, generates critique text) and $r_\psi = h_r \circ M_\theta$ (a randomly initialised scalar head, predicts reward). q_ϕ plays a dual role: it is simultaneously the *approximate posterior* over the latent critiques z (pushed toward the oracle p^* by ℓ_c) and the generator of critique samples that r_ψ conditions on in ℓ_r.

Introducing critiques for chosen and rejected responses z^+, z^- as latents and marginalising:

$$p(y^+ \succ y^- \mid x) = \sum_{z^+, z^-} p(y^+ \succ y^- \mid z^+, z^-, x)\, p^*(z^+ \mid y^+, x)\, p^*(z^- \mid y^-, x)$$

Since p^* is intractable, substitute the learned q_ϕ and apply Jensen's inequality to obtain the ELBO:

$$\log p(y^+ \succ y^- \mid x) \;\geq\; \mathbb{E}_{q_\phi(z^+|y^+,x),\, q_\phi(z^-|y^-,x)} \left[\log \frac{p(y^+ \succ y^-,\, z^+, z^- \mid x)}{q_\phi(z^+|y^+, x)\, q_\phi(z^-|y^-, x)}\right] \tag{27}$$

Maximising this ELBO decomposes into two objectives. The **critique generation loss** ℓ_c trains q_ϕ to approximate p^*. Reverse KL would cause entropy collapse, so *forward* KL is used, treating the filtered critique set Z as a uniform proxy for p^*:

$$\ell_c = -\frac{1}{|Z|} \sum_{z \in Z} \log q_\phi(z \mid y, x) \tag{28}$$

The **preference modeling loss** ℓ_r trains r_ψ: the critique z is appended to the response, and the scalar head pools from the concatenation $[y; z]$:

$$\ell_r = -\log \sigma\big(r_\psi(x, [y^+; z^+]) - r_\psi(x, [y^-; z^-])\big) \tag{29}$$

Dynamic weighting schedule. The two losses are combined with a time-varying weight $\lambda(t)$:

$$\mathcal{L}(\phi, \psi) = \lambda(t) \cdot \ell_c + \big(1 - \lambda(t)\big) \cdot \ell_r, \qquad \lambda(t) = \begin{cases} 1 & t < (K-1)T \\ 1 - \beta \cdot \dfrac{t - (K-1)T}{T} & t \geq (K-1)T \end{cases} \tag{30}$$

With $K = 2$ epochs and $\beta = 0.9$: the first epoch trains *only* on critique generation ($\lambda = 1$, reward head receives no gradient); in the final epoch λ decays from 1 to 0.1, gradually shifting supervision to reward prediction. This ordering is essential — the reward head only begins training once the LM already has strong critique capability, preventing reward-head overfitting before useful critiques can be generated. At inference, m critiques are sampled and scalar rewards averaged: $\bar{r} = \frac{1}{m}\sum_{i=1}^{m} r_\psi(x, [y, z_i])$. Critic-RM at 10% of training data (87.7) already outperforms the standard RM at 100% data (86.4), demonstrating that jointly trained self-critiques are substantially more data-efficient than plain preference labels.

3.4 Training the Base Model with Critique

The methods in Section 3.3 improve reward models and judges by having them generate critique. A closely related cluster instead trains the base *policy* model directly using critique signals. This section covers the methods most directly related to CDPO.

3.4.1 Critique Fine-Tuning

[Y. Wang et al., 2025] ask: what happens if we use *only* the NLL term on critique data, without any preference optimisation? In Critique Fine-Tuning (CFT), the base model is trained on the teacher's critique c_v of a given query-response pair by maximising:

$$\mathcal{L}_{\text{CFT}}(\theta) = -\sum_{t=1}^{|c_v|} \log \pi_\theta(c_{v,t} \mid c_{v,<t}, [x; y]) \tag{31}$$

where $[x; y]$ concatenates the query and a (potentially flawed) response, and c_v is a critique generated by a teacher model (GPT-4o). At inference, the CFT-trained model generates direct answers without producing critiques.

3.4.2 SuperCorrect: Cross-Model Collaborative DPO

[Yang et al., 2024] introduce a two-stage framework. Stage 1 distils hierarchical thought templates from a large teacher model (o1-preview) via SFT. Stage 2 applies **cross-model collaborative DPO** on teacher-guided error-correction traces. Given an erroneous student solution with the first error at step k, the teacher generates a correction trace (a_k^+, c_k^+) (error analysis and correction guidance) while the student independently generates (a_k^-, c_k^-). The DPO loss is:

$$\mathcal{L}_{\text{Cross-DPO}} = -\mathbb{E}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(a_k^+, c_k^+ \mid x, s_{<k})}{\pi_{\text{ref}}(a_k^+, c_k^+ \mid x, s_{<k})} - \beta \log \frac{\pi_\theta(a_k^-, c_k^- \mid x, s_{<k})}{\pi_{\text{ref}}(a_k^-, c_k^- \mid x, s_{<k})}\right)\right] \tag{32}$$

where $s_{<k}$ are the correct preceding steps. The teacher's correction trace plays the role of c_v in CDPO. SuperCorrect-7B achieves 70.2% on MATH (surpassing DeepSeekMath-7B by 7.8%) and demonstrates that cross-model DPO at the step level substantially improves self-correction capabilities. CriticGPT [McAleese et al., 2024] and CTRL [Xie et al., 2025] provide complementary evidence that LLMs can be explicitly taught to generate high-quality critiques.

3.5 Positioning of CDPO

The three defining features of CDPO are: **(A)** a DPO loss on answer preferences (y_w, y_l); **(B)** an NLL term on the verifier's CoT critique c_v; and **(C)** applied to the base policy model. No single prior work combines all three.

Paper	A: DPO on answers	B: NLL on verifier CoT	C: Base model
CDPO (this work)	✓	✓	✓
IRPO [Pang et al., 2024]	✓	NLL on own CoT	✓
AIPO / RPO [Shen et al., 2024]; [Z. Liu et al., 2024]	✓	NLL on y_w	✓
DJPO [P. Wang et al., 2024]	✓	✓	× (judge)
GenRM-STaR-DPO [Mahan et al., 2024]	✓	✓	× (judge)
CFT [Y. Wang et al., 2025]	×	✓	✓
SuperCorrect [Yang et al., 2024]	On traces	Teacher correction	✓

CDPO sits at the intersection of three lines of work that have so far only been combined in pairs: (1) IRPO establishes the right loss structure (DPO + NLL) but anchors to the model's own CoT rather than an external verifier's critique; (2) DJPO and GenRM-STaR-DPO use verifier reasoning as the NLL target — the right data — but apply it to train a standalone judge rather than the answer-generating base policy; and (3) CFT applies verifier critique as a direct NLL target for the base policy — the right model and signal — but omits the DPO preference term entirely.

4 Methodology

We now describe the Critique DPO (CDPO) objective, and the dataset construction, and specify the baseline methods against which CDPO is compared.

4.1 Data Construction

We use the GenRM dataset [Zhang et al., 2024], which provides math problems from the GSM8K [Cobbe et al., 2021] training set. For each problem x, multiple candidate solutions are generated by Gemini 1.0 Pro. Each candidate solution is labelled correct or incorrect by checking the extracted final answer against the ground-truth answer. An LLM oracle generates a step-by-step verification rationale *for each individual solution*, verifying each reasoning step against the expected answer and concluding with a "Yes" (correct) or "No" (incorrect) judgement. We denote the verification rationale for a correct solution as $c_{v,w}$ and for an incorrect solution as $c_{v,l}$.

From this raw data, we construct preference triples (x, y_w, y_l, c_v) by pairing, for each problem, a correct solution y_w with an incorrect solution y_l. The critique sequence c_v is then assembled by concatenating *both* solutions together with their respective verification rationales into a single contrastive narrative:

$c_v =$ "This is a correct solution and preferred:" $+ y_w +$ "Here's why this solution is correct and preferred:" $+ c_{v,w} +$ "This is an incorrect solution and not preferred:" $+ y_l$ "Here's why this solution is incorrect and not preferred:" $+ c_{v,l}$

We then derive four training-set variants:

- **CDPO**: each sample is $(x,\ y_w,\ y_l,\ c_v)$ — a preference pair with the composite critique.
- **DPO**: each sample is $(x,\ y_w,\ y_l)$ — the same preference pairs, without any verification reasoning.
- **SFT (no verification)**: each sample is $(x,\ y_w)$ — supervised fine-tuning on correct solutions only.
- **SFT (with verification)**: each sample is $(x,\ y_w,\ c_{v,w})$ — supervised fine-tuning on the correct solution concatenated with its individual verification rationale $c_{v,w}$

4.2 CDPO: Critique Direct Preference Optimization

CDPO augments the standard DPO loss with an auxiliary negative log-likelihood (NLL) term that trains the base policy to reproduce the verifier's chain-of-thought critique. The combined loss is

$$\mathcal{L}_{\text{CDPO}}(\theta) = (1-\alpha)\,\mathcal{L}_{\text{DPO}}(\theta) \;+\; \alpha\,\mathcal{L}_{\text{critique}}(\theta) \tag{33}$$

where $\alpha \in [0, 1]$ controls the trade-off between the preference signal and the critique signal. The DPO term $\mathcal{L}_{\text{DPO}}$ is the standard loss from Eq. (5). The critique term is the length-normalised NLL on the

verifier's chain-of-thought:

$$\mathcal{L}_{\text{critique}}(\theta) = -\frac{1}{|c_v|} \sum_{t=1}^{|c_v|} \log \pi_\theta(c_{v,t} \mid c_{v,<t},\, x) \tag{34}$$

The composite critique c_v is a contrastive narrative: it restates both the preferred and less preferred solutions, then provides step-by-step verification rationales that trace where each solution's reasoning succeeds or fails. Concretely, the model receives the input sequence $[x;\, c_v]$ — the problem followed by the critique — and the NLL loss is computed only over the c_v tokens. Because c_v embeds both solutions y_w and y_l within its narrative, it is self-contained: the model learns to generate the full contrastive analysis given only the problem statement. By training the base policy to reproduce this analysis, we hypothesise that the model will learn to distinguish correct from incorrect reasoning steps, thereby improving its own solution quality.

CDPO instantiates the generic DPO+NLL template of Eq. (8) with the NLL target set to the external verifier's critique c_v, rather than the model's own chain-of-thought (as in IRPO; Section 3.2.1) or the winning response y_w (as in AIPO/RPO; Sections 3.2.2, 3.2.3). Unlike DJPO and GenRM-STaR-DPO (Sections 3.2.4, 3.3.2), which also use verifier reasoning as the NLL target but train a standalone judge, CDPO applies the critique signal directly to the answer-generating base policy.

4.3 Baseline Methods

We compare CDPO against three baselines, all trained on the same underlying data:

DPO. The standard DPO loss (Eq. 5), equivalent to setting $\alpha = 0$ in Eq. (33). This isolates the contribution of the critique term.

SFT (no verification). Standard supervised fine-tuning on correct solutions:

$$\mathcal{L}_{\text{SFT}}(\theta) = -\sum_{t=1}^{|y_w|} \log \pi_\theta(y_{w,t} \mid y_{w,<t},\, x) \tag{35}$$

This establishes a lower bound: what can be achieved without any preference signal.

SFT (with verification). The same SFT objective, but the training target is the concatenation $[y_w + c_v]$ — the correct solution followed by the verification rationale. This tests whether the verification data is useful as a pure SFT signal, without the DPO preference term.

5 Experimental Setup

5.1 Training Configuration

We use Qwen2.5-0.5B-Instruct [Qwen-Team, 2024] as the base model. All four methods share the same base configuration: LoRA with rank 8 applied to all linear modules, learning rate 1×10^{-6} with cosine decay and 10% warmup, 2 training epochs, bf16 mixed precision, FlashAttention-2, and DeepSpeed ZeRO Stage 3 across 8 NVIDIA H100 80GB GPUs. Method-specific settings are summarised in Table 1.

Table 1: Method-specific hyperparameters. All other settings (LoRA rank, learning rate, scheduler, epochs, precision) are shared across methods.

	CDPO	**DPO**	**SFT-veri**	**SFT-no_veri**
Batch size / GPU	6	16	32	48
β	0.1	0.1	—	—
α (critique weight)	0.5	—	—	—
Length-normalised critique	Yes	—	—	—
Gradient checkpointing	Yes	No	Yes	Yes

The batch sizes differ across methods because CDPO processes three sequences per example (chosen, rejected, critique), requiring significantly more memory than the other methods.

5.2 Evaluation

We evaluate on two mathematical reasoning benchmarks: GSM8K [Cobbe et al., 2021] (500 test samples) and MATH-500 [Hendrycks et al., 2021]. All evaluations use greedy decoding (temperature 0, top-$p = 1$) via vLLM for efficient inference. LoRA adapters are merged with the base model before evaluation. We evaluate checkpoints saved at every 10% of training (yielding 10–14 checkpoints per method), reporting both the best checkpoint accuracy and the final checkpoint accuracy to capture training dynamics.

6 Results

6.1 Main Results

Table 2 summarises the best and final checkpoint accuracy for each method. The baseline row shows the pre-training accuracy of the unmodified Qwen2.5-0.5B-Instruct model (checkpoint 0, shared across all methods).

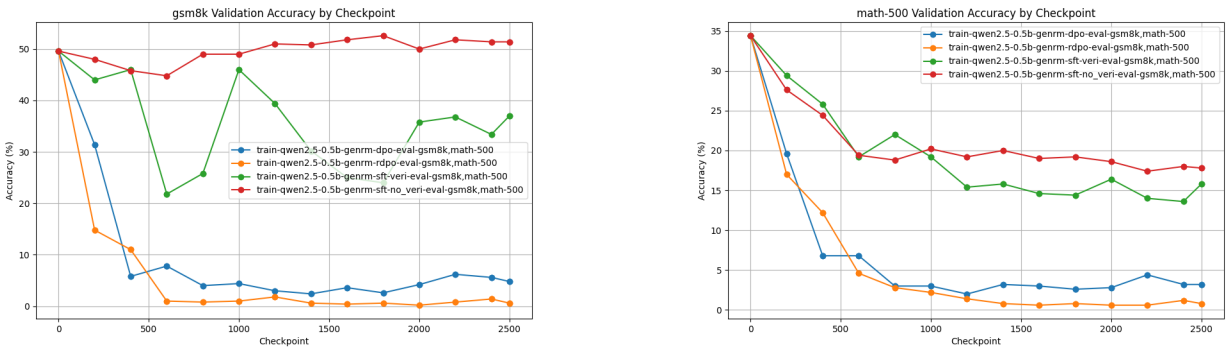
Table 2: Test accuracy (%) on GSM8K and MATH-500. *Best* reports the highest accuracy across all checkpoints; *Final* reports the last checkpoint. The baseline is the unmodified Qwen2.5-0.5B-Instruct model.

	GSM8K		**MATH-500**	
Method	Best	Final	Best	Final
Baseline (ckpt-0)	49.6	—	34.4	—
SFT (no verification)	**51.8**	51.4	**35.4**	32.6
SFT (with verification)	50.8	50.8	35.2	33.0
DPO	49.6	4.8	34.4	3.2
CDPO ($\alpha = 0.5$)	49.6	0.6	34.4	0.8

SFT methods provide modest, stable improvements over the baseline, while both preference-based methods (DPO and CDPO) suffer catastrophic performance collapse. Critically, CDPO — the method designed to leverage richer feedback — performs *worse* than vanilla DPO.

6.2 Training Dynamics

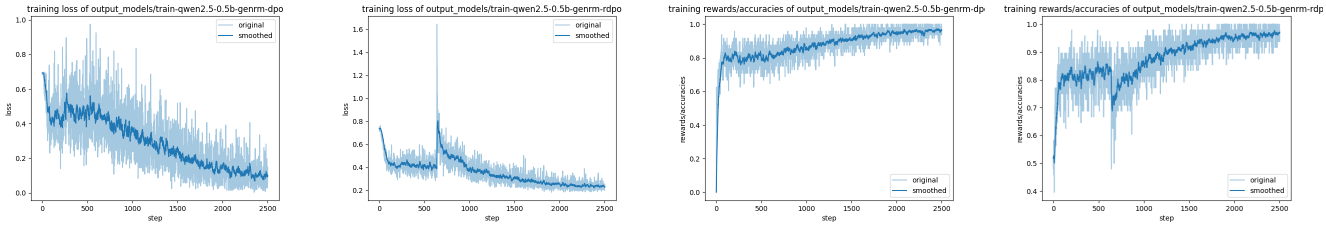
Figure 1 shows evaluation accuracy across training checkpoints. The SFT variants maintain stable accuracy throughout training, with SFT (no verification) reaching 51.8% on GSM8K — a modest 2.2 percentage point improvement over the 49.6% baseline. Both DPO and CDPO begin at the baseline accuracy (checkpoint 0 is the unmodified model) and collapse rapidly: DPO drops to ∼6% within the first 400 training steps, while CDPO collapses even faster to ∼1% by step 600. Neither method recovers.



(a) GSM8K (b) MATH-500

Figure 1: Evaluation accuracy over training checkpoints for all four methods. DPO and CDPO collapse rapidly from the baseline, while both SFT variants remain stable. CDPO collapses faster and to a lower floor than DPO.

Figure 2 reveals the mechanism behind this collapse. Despite the catastrophic drop in downstream accuracy, both DPO and CDPO show *smoothly decreasing training loss* and *reward accuracy approaching 1.0* throughout training. This is the classic signature of reward hacking: the model learns to separate chosen from rejected examples in implicit-reward space (by pushing log-probability ratios apart), but this optimisation does not correspond to improved generation quality — it instead degrades the model's ability to produce coherent solutions.



(a) DPO loss (b) CDPO loss (c) DPO reward acc. (d) CDPO reward acc.

Figure 2: Training metrics for DPO and CDPO. Both methods show smoothly decreasing loss (a, b) and reward accuracy approaching 1.0 (c, d), despite the catastrophic collapse in downstream evaluation accuracy shown in Figure 1. This divergence between training metrics and actual performance is the hallmark of reward hacking.

7 Discussion

7.1 DPO Collapse and Reward Hacking

The dramatic collapse of both DPO and CDPO, combined with smoothly improving training metrics, is a well-documented failure mode of DPO. [H. Liu et al., 2025] identify exactly this pattern in text-to-SQL: without chain-of-thought data, the implicit reward $\hat{r}_\theta$ can be inflated by superficial correlates that do not reflect reasoning quality — model performance initially peaks then decreases, while the implicit reward continues to rise. Our results confirm this finding in the mathematical reasoning domain.

At the 0.5B parameter scale, the model appears to lack sufficient capacity to simultaneously maintain generation quality and satisfy the DPO preference margin. The KL constraint (controlled by $\beta = 0.1$) is evidently insufficient to prevent the policy from drifting into a degenerate region of the output space.

7.2 CDPO Exacerbates Collapse

Rather than mitigating the DPO collapse as hypothesised, the critique NLL term in CDPO *accelerates* it. CDPO's final-checkpoint accuracy on GSM8K (0.6%) is an order of magnitude worse than DPO's (4.8%). We identify two potential mechanisms:

Distributional mismatch. The verification rationale c_v is generated by an external oracle whose writing style, reasoning depth, and vocabulary differ substantially from the base model's natural distribution. This echoes findings from GenRM-STaR-DPO [Mahan et al., 2024], where off-policy rationales from a stronger model (GPT-4 or Llama-3.1-70B) matched STaR-DPO in-distribution but collapsed out-of-distribution. CLoud [Ankner et al., 2024] similarly reports a 5.6% drop when training on oracle critiques versus self-generated ones, attributing it to distributional shift between training and inference.

Task conflict. The verification rationale is fundamentally different from the solution-generation task the model performs at inference. Verification text contains meta-analytical language ("Step 1: check...", "Expected Answer: Correct", "Verification: Is the answer correct? Yes/No") that is absent from problem-solving. Training the model to produce both types of text with a single set of LoRA parameters creates a multi-task conflict that degrades performance on the primary task. This is consistent with the observation from CFT [Y. Wang et al., 2025] that models trained on critique data generate direct answers at inference *without producing critiques* — the critique knowledge transfers implicitly. In our case, with the competing DPO gradient, this implicit transfer may be disrupted.

7.3 Why Verification Data Does Not Help SFT Either

Both SFT variants perform comparably, with SFT (no verification) slightly ahead. This suggests that, at the 0.5B scale, the model cannot usefully extract signal from verification-style text when its inference-time task is solution generation. The verification rationale is informative about *what makes a solution correct*, but this information does not straightforwardly transfer to *producing correct solutions*. This aligns with the broader finding that verification and generation are distinct capabilities that may require separate training [Hosseini et al., 2024].

8 Conclusion

We proposed Critique DPO (CDPO), which extends the DPO objective with an auxiliary NLL term on a verifier's chain-of-thought critique. We evaluated CDPO against DPO and SFT baselines on Qwen2.5-0.5B-Instruct using 120,000 math preference triples from the GenRM dataset, with evaluation on GSM8K and MATH-500.

Our results are a negative finding: both DPO and CDPO suffer catastrophic performance collapse, with CDPO collapsing faster and more severely than vanilla DPO. The critique NLL term, intended to transfer the verifier's analytical capabilities to the base model, instead exacerbates the well-documented DPO reward-hacking failure mode. Meanwhile, simple SFT on correct solutions provides modest but stable improvements, and adding verification rationales to SFT yields no additional benefit.

These results contribute to the growing understanding of when and how auxiliary reasoning signals can improve fine-tuning. Our findings suggest that verifier critique is most effective when used to train models that will *produce* critiques (judges and verifiers), rather than models that will produce solutions (base policies). Future work should explore: (1) on-policy critique generation (STaR-style self-generated rationales instead of off-policy oracle critiques), (2) larger model scales where the multi-task learning implicit in CDPO may become tractable, (3) iterative reference model updates to prevent the distributional drift that drives collapse, and (4) curriculum scheduling of α to reduce the critique weight as training progresses.

A CDPO Training Algorithm

Algorithm 1 presents the pseudocode for CDPO training, corresponding to the implementation in `CustomCDPOT`

Algorithm 1 CDPO Training (per batch)

Require: Policy π_θ, reference π_{ref}, batch $\{(x_i, y_{w,i}, y_{l,i}, c_{v,i})\}_{i=1}^{B}$, weights α, β

Concatenate chosen, rejected, and critique into single batch of size $3B$
logits $\leftarrow \pi_\theta(\text{batch})$ ▷ Single forward pass, shape $3B \times T \times V$
all_logps, lengths $\leftarrow$ GETBATCHLOGPS(logits, labels) ▷ Shape $3B$
Delete logits; free GPU memory
$\log \pi_\theta(y_w), \log \pi_\theta(y_l), \log \pi_\theta(c_v) \leftarrow$ SPLIT(all_logps, B)
$L_w, L_l, L_c \leftarrow$ SPLIT(lengths, B) ▷ Valid token counts

// Reference model log-probs (LoRA: disable adapter)
$\log \pi_{\text{ref}}(y_w), \log \pi_{\text{ref}}(y_l) \leftarrow$ same procedure with π_{ref}

// Implicit rewards
$\hat{r}_w \leftarrow \beta\,(\log \pi_\theta(y_w) - \log \pi_{\text{ref}}(y_w))$
$\hat{r}_l \leftarrow \beta\,(\log \pi_\theta(y_l) - \log \pi_{\text{ref}}(y_l))$

$\mathcal{L}_{\text{DPO}} \leftarrow -\log \sigma(\hat{r}_w - \hat{r}_l)$
$\mathcal{L}_{\text{critique}} \leftarrow -\log \pi_\theta(c_v) \;/\; L_c$ ▷ Length-normalised NLL
$\mathcal{L}_{\text{CDPO}} \leftarrow \text{mean}\big[(1-\alpha)\,\mathcal{L}_{\text{DPO}} + \alpha\,\mathcal{L}_{\text{critique}}\big]$
return $\mathcal{L}_{\text{CDPO}}$

B Data Processing Pipeline

The data processing pipeline consists of three stages:

Stage 1: Download and extract. The raw GenRM dataset is downloaded from HuggingFace (`flowingpurple/critiques` split). Each entry in the raw dataset contains: an `inputs` field (the problem prompt concatenated with a model-generated solution and a verification prompt), a `targets` field (the LLM oracle's step-by-step verification rationale for that specific solution), a `target` field (the ground-truth numerical answer), and metadata (`question_id`, `model_output_id`). We extract from `inputs`: (a) the math problem (text between `\nQ:` and `\nA:`), and (b) the model-generated solution (text between `\nA:` and `\nVerification`). Each entry is labelled correct or incorrect by extracting the final answer from the solution and comparing against `target`.

Stage 2: Pair. We filter for problems (`question_ids`) that have *both* at least one correct and at least one incorrect solution. For each eligible problem, we pair one correct solution with one incorrect solution (without replacement, so each `model_output_id` is used at most once). We then randomly sample pairs to reach the target dataset size (120,000 pairs from 6,452 eligible problems). The resulting "master" dataset has alternating correct/incorrect rows.

Stage 3: Format into variants. From each (correct, incorrect) pair in the master dataset, we produce four dataset formats:

- **CDPO**: The composite critique c_v is constructed by concatenating both solutions and both of their individual verification rationales (from the `targets` column) with connecting phrases ("This is a correct solution and preferred:", etc.), as described in Section 4.1.
- **DPO**: The chosen and rejected solutions only, without any verification data.
- **SFT (no verification)**: Only the correct solution is kept, formatted as instruction/input/output.
- **SFT (with verification)**: The correct solution is concatenated with its individual verification rationale $c_{v,w}$ (from the correct entry's `targets` column), preceded by "Verification: Let's verify step by step."

C Full Checkpoint Results

Tables 3–6 report the complete checkpoint-by-checkpoint evaluation results.

Table 3: DPO: accuracy (%) at each checkpoint.

Checkpoint	GSM8K	MATH-500
0	49.6	34.4
200	31.4	19.6
400	5.8	6.8
600	7.8	6.8
800	4.0	3.0
1000	4.4	3.0
1200	3.0	2.0
1400	2.4	3.2
1600	3.6	3.0
1800	2.6	2.6
2000	4.2	2.8
2200	6.2	4.4
2400	5.6	3.2
2500	4.8	3.2

Table 4: CDPO ($\alpha = 0.5$): accuracy (%) at each checkpoint.

Checkpoint	GSM8K	MATH-500
0	49.6	34.4
200	14.8	17.0
400	11.0	12.2
600	1.0	4.6
800	0.8	2.8
1000	1.0	2.2
1200	1.8	1.4
1400	0.6	0.8
1600	0.4	0.6
1800	0.6	0.8
2000	0.2	0.6
2200	0.8	0.6
2400	1.4	1.2
2500	0.6	0.8

Table 5: SFT (no verification): accuracy (%) at each checkpoint.

Checkpoint	GSM8K	MATH-500
0	49.6	34.4
63	47.0	31.0
126	49.6	35.2
189	50.0	32.6
252	50.4	34.2
315	51.0	33.2
378	51.2	34.0
441	51.4	35.0
504	51.4	34.4
567	50.0	35.4
626	51.8	32.6

Table 6: SFT (with verification): accuracy (%) at each checkpoint.

Checkpoint	GSM8K	MATH-500
0	49.6	34.4
94	48.8	35.0
188	47.8	33.2
282	50.8	35.2
376	50.8	32.8
470	50.4	32.8
564	50.4	35.0
658	49.8	33.0
752	50.2	32.6
846	49.6	34.4
938	50.8	33.0

D Hyperparameter Details

Table 7 lists all training hyperparameters.

Table 7: Complete training hyperparameters for all methods.

Parameter	CDPO	DPO	SFT-veri	SFT-no_veri
Base model	Qwen2.5-0.5B-Instruct			
Fine-tuning method	LoRA (rank 8, all linear modules)			
Learning rate	1×10^{-6}			
LR scheduler	Cosine with 10% warmup			
Training epochs	2			
Precision	bf16			
Attention	FlashAttention-2			
Parallelism	DeepSpeed ZeRO Stage 3, 8× H100			
Batch size / GPU	6	16	32	48
Cutoff length (tokens)	1664	640	1024	768
β (DPO temperature)	0.1	0.1	—	—
α (critique weight)	0.5	—	—	—
Normalise critique	Yes	—	—	—
Preference loss	sigmoid	sigmoid	—	—
Gradient checkpointing	Yes	No	Yes	Yes
Dataset size	120K	120K	120K	120K
Save frequency	10%	10%	10%	10%

E Training Systems and Infrastructure

This appendix describes the systems-level techniques that made training feasible on our hardware. We cover parameter-efficient fine-tuning (LoRA), mixed-precision arithmetic, memory-efficient attention, and distributed training via DeepSpeed ZeRO. These are standard techniques, but understanding their interaction is essential for reproducing our results and for appreciating the memory constraints that shaped our experimental design — particularly the batch size limitations for CDPO.

E.1 Low-Rank Adaptation (LoRA)

Full fine-tuning updates all $|\theta|$ parameters of the model, requiring storage of the full gradient and optimiser states ($2|\theta|$ floats for Adam's first and second moments). For a 0.5B-parameter model in bf16, the model weights alone occupy ∼1 GB, but Adam states in fp32 add ∼8 GB.

LoRA freezes the pretrained weights $W_0 \in \mathbb{R}^{d\times k}$ and injects a low-rank update:

$$W = W_0 + \Delta W = W_0 + BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d, k)$$

Only A and B are trained. The number of trainable parameters is $r(d + k)$ per adapted layer, versus dk for full fine-tuning. With rank $r = 8$ applied to all linear modules of Qwen2.5-0.5B-Instruct, the trainable parameter count is ∼1.6M — roughly 0.3% of the full model.

A scaling factor $\frac{\alpha_{\text{LoRA}}}{r}$ is applied to the low-rank update, where α_{LoRA} controls the magnitude (we use $\alpha_{\text{LoRA}} = 2r = 16$). At initialisation, A is drawn from a Gaussian and B is set to zero, so $\Delta W = 0$ and the model begins from the pretrained checkpoint.

Reference model in LoRA. DPO and CDPO require a frozen reference model π_{ref}. With LoRA, the reference is the base model *without* the adapter — obtained by temporarily disabling the LoRA layers during the reference forward pass. This avoids loading a second copy of the model into GPU memory, effectively halving the memory cost of the reference model compared to full fine-tuning.

E.2 Mixed-Precision Training (bf16)

We train in bfloat16 (bf16) mixed precision. bf16 uses 1 sign bit, 8 exponent bits, and 7 mantissa bits, providing the same dynamic range as fp32 ($\sim 10^{\pm 38}$) but with reduced precision ($\sim$3 decimal digits vs $\sim$7). Compared to fp16 (5 exponent bits, 10 mantissa bits), bf16 trades mantissa precision for a much larger exponent range, eliminating the overflow and loss-scaling issues that plague fp16 training.

The memory savings are straightforward: each parameter and activation occupies 2 bytes instead of 4, halving the memory footprint of the forward and backward passes. The Adam optimiser states (first and second moments) are maintained in fp32 for numerical stability, but gradients and activations are computed and communicated in bf16.

In our implementation, we explicitly avoid upcasting the logits tensor to fp32 during the CDPO forward pass (the default in some DPO implementations). Since the logits tensor has shape $3B \times T \times V$ and the vocabulary size $V = 151{,}936$ for Qwen2.5, this tensor alone can occupy several gigabytes. Keeping it in bf16 approximately halves this cost, which is critical for CDPO where the batch dimension is $3\times$ larger than standard DPO.

E.3 FlashAttention-2

Standard self-attention computes $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$, materialising the full $T \times T$ attention matrix in GPU memory. For sequence length T, this requires $\mathcal{O}(T^2)$ memory and $\mathcal{O}(T^2 d)$ compute.

FlashAttention-2 computes the *exact same result* without materialising the full attention matrix, by using a tiled algorithm that operates on blocks of Q, K, V that fit in GPU SRAM (on-chip memory). The key insight is that softmax can be decomposed into local block-wise computations with a running correction for the normalisation constant (online softmax). The memory complexity drops to $\mathcal{O}(T)$ and the wall-clock time is reduced by 2–4$\times$ due to fewer HBM (off-chip memory) reads.

For our experiments with cutoff lengths up to 1664 tokens, FlashAttention-2 reduces the per-layer attention memory from $T^2 \cdot \text{num_heads} \cdot \text{sizeof(bf16)}$ (quadratic) to linear in T, freeing memory for larger batch sizes.

E.4 Gradient Checkpointing

During the backward pass, computing gradients requires the intermediate activations from the forward pass. Naively, all activations are stored, giving memory cost $\mathcal{O}(L \cdot B \cdot T \cdot d)$ where L is the number of layers. Gradient checkpointing trades compute for memory: only a subset of activations (typically at layer boundaries) are stored, and the intermediate activations are *recomputed* during the backward pass from the nearest checkpoint.

For a transformer with L layers, checkpointing at every layer boundary reduces activation memory from $\mathcal{O}(L)$ to $\mathcal{O}(\sqrt{L})$ per layer's internal activations (since only one layer's activations need to be live at a time), at the cost of one additional forward pass per layer. In practice this adds $\sim$30% wall-clock overhead but can reduce activation memory by 60–70%.

We enable gradient checkpointing for CDPO and both SFT variants. For DPO, the smaller batch size and shorter sequences meant gradient checkpointing was not strictly necessary; we disabled it to reduce training time.

E.5 DeepSpeed ZeRO Stage 3

Training is distributed across 8 $\times$ H100 80 GB GPUs using DeepSpeed ZeRO (Zero Redundancy Optimizer) Stage 3. Without ZeRO, data-parallel training replicates the full model, gradients, and optimiser states on every GPU — a $3\times$ memory redundancy per parameter.

ZeRO eliminates this redundancy by *partitioning* model states across GPUs:

- **Stage 1** partitions optimiser states (Adam moments). Each GPU stores $1/N$ of the optimiser state (N = number of GPUs). Memory per GPU: model + gradients + $\frac{1}{N}$ optimiser.
- **Stage 2** additionally partitions gradients. Memory per GPU: model + $\frac{1}{N}$ (gradients + optimiser).
- **Stage 3** additionally partitions model parameters. Memory per GPU: $\frac{1}{N}$ (model + gradients + optimiser).

With Stage 3, the memory per GPU scales as $\mathcal{O}(|\theta|/N)$ rather than $\mathcal{O}(|\theta|)$. The cost is increased communication: parameters must be gathered (all-gathered) before each forward and backward computation, and gradients must be reduce-scattered after each backward step. The `overlap_comm: true` setting in our DeepSpeed configuration overlaps these communications with computation to hide latency.

For our 0.5B model on 8 GPUs, Stage 3 reduces the per-GPU model state memory from $\sim$4 GB (model + optimiser in fp32) to $\sim$0.5 GB, freeing the remaining $\sim$79 GB for activations, the logits tensor, and batch data. This is essential for CDPO, where the $3B \times T \times V$ logits tensor is the dominant memory consumer.

E.6 Memory Analysis for CDPO

The peak memory bottleneck in CDPO is the logits tensor produced by the forward pass. With batch size $B = 6$ per GPU, three sequences per example (chosen, rejected, critique), maximum sequence length $T = 1664$, and vocabulary size $V = 151{,}936$:

$$\text{Logits memory} = 3B \times T \times V \times 2 \text{ bytes} = 18 \times 1664 \times 151{,}936 \times 2 \approx 8.6 \text{ GB}$$

This single tensor consumes over 10% of the GPU's 80 GB HBM. After extracting the per-token log-probabilities via `get_batch_logps()` (which reduces to a $3B$-dimensional vector), we immediately delete the logits tensor and call `torch.cuda.empty_cache()` to return this memory to the allocator. Without this explicit deallocation, PyTorch's caching allocator would retain the memory, potentially causing OOM errors during the reference model's forward pass (which produces a second logits tensor of the same size).

The combination of all techniques — LoRA (small trainable parameter set), bf16 (halved activation memory), FlashAttention-2 (linear attention memory), gradient checkpointing (reduced activation storage), ZeRO-3 (partitioned model states), and explicit logits deletion — was necessary to fit CDPO training within the 80 GB per-GPU budget. Even so, the batch size was limited to 6 per GPU (vs. 16 for DPO and 48 for SFT), reflecting the $3\times$ memory multiplier from processing three sequences per example.

References

Rafailov, Rafael et al. (2024). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. arXiv: 2305.18290 [cs.LG]. URL: https://arxiv.org/abs/2305.18290.

Wolfe, Cameron R. (2025). *Direct Preference Optimization (DPO)*. Blog post. URL: https://cameronrwolfe.substack.com/p/direct-preference-optimization.

Romero, Tyler (2024). *Direct Preference Optimization Explained In-depth*. Blog post. URL: https://www.tylerromero.com/posts/2024-04-dpo/.

Bai, Yuntao et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv: 2212.08073 [cs.CL]. URL: https://arxiv.org/abs/2212.08073.

Lee, Harrison et al. (2024). *RLAIF vs. RLHF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. arXiv: 2309.00267 [cs.CL]. URL: https://arxiv.org/abs/2309.00267.

Li, Ang et al. (2024). *HRLAIF: Improvements in Helpfulness and Harmlessness in Open-domain Reinforcement Learning From AI Feedback*. arXiv: 2403.08309 [cs.LG]. URL: https://arxiv.org/abs/2403.08309.

Liu, Hanbing et al. (2025). *Uncovering the Impact of Chain-of-Thought Reasoning for Direct Preference Optimization*. arXiv: 2502.11656 [cs.CL]. URL: https://arxiv.org/abs/2502.11656.

Zelikman, Eric et al. (2022). *STaR: Bootstrapping Reasoning With Reasoning*. arXiv: 2203.14465 [cs.LG]. URL: https://arxiv.org/abs/2203.14465.

Hosseini, Arian et al. (2024). *V-STaR: Training Verifiers for Self-Taught Reasoners*. arXiv: 2402.06457 [cs.LG]. URL: https://arxiv.org/abs/2402.06457.

Yuan, Weizhe et al. (2024). *Self-Rewarding Language Models*. arXiv: 2401.10020 [cs.CL]. URL: https://arxiv.org/abs/2401.10020.

Pang, Richard Yuanzhe et al. (2024). *Iterative Reasoning Preference Optimization*. arXiv: 2404.19733 [cs.CL]. URL: https://arxiv.org/abs/2404.19733.

Shen, Yaojie et al. (2024). *AIPO: Improving Training Objective for Iterative Preference Optimization*. arXiv: 2409.08845 [cs.CL]. URL: https://arxiv.org/abs/2409.08845.

Liu, Zhihan et al. (2024). *Provably Mitigating Overoptimization in RLHF: Your SFT Loss is Implicitly an Adversarial Regularizer*. arXiv: 2405.16436 [cs.LG]. URL: https://arxiv.org/abs/2405.16436.

Wang, Peifeng et al. (2024). *Direct Judgement Preference Optimization*. arXiv: 2409.14664 [cs.CL]. URL: https://arxiv.org/abs/2409.14664.

Zhang, Lunjun et al. (2024). *Generative Verifiers: Reward Modeling as Next-Token Prediction*. arXiv: 2408.15240 [cs.LG]. URL: https://arxiv.org/abs/2408.15240.

Mahan, Dakota et al. (2024). *Generative Reward Models*. arXiv: 2410.12832 [cs.LG]. URL: https://arxiv.org/abs/2410.12832.

Ankner, Zachary et al. (2024). *Critique-out-Loud Reward Models*. arXiv: 2408.11791 [cs.LG]. URL: https://arxiv.org/abs/2408.11791.

Yu, Yue et al. (2025). *Self-Generated Critiques Boost Reward Modeling for Language Models*. arXiv: 2411.16646 [cs.CL]. URL: https://arxiv.org/abs/2411.16646.

Wang, Yubo, Xiang Yue, and Wenhu Chen (2025). *Critique Fine-Tuning: Learning to Critique is More Effective than Learning to Imitate*. arXiv: 2501.17703 [cs.CL]. URL: https://arxiv.org/abs/2501.17703.

Yang, Ling et al. (2024). *SuperCorrect: Advancing Small LLM Reasoning with Thought Template Distillation and Self-Correction*. arXiv: 2410.09008 [cs.CL]. URL: https://arxiv.org/abs/2410.09008.

McAleese, Nat et al. (2024). *LLM Critics Help Catch LLM Bugs*. arXiv: 2407.00215 [cs.CL]. URL: https://arxiv.org/abs/2407.00215.

Xie, Zhihui et al. (2025). *Teaching Language Models to Critique via Reinforcement Learning*. arXiv: 2502.03492 [cs.CL]. URL: https://arxiv.org/abs/2502.03492.

Cobbe, Karl et al. (2021). *Training Verifiers to Solve Math Word Problems*. arXiv: 2110.14168 [cs.LG].

Qwen-Team (2024). *Qwen2.5 Technical Report*. arXiv: 2412.15115 [cs.CL].

Hendrycks, Dan et al. (2021). *Measuring Mathematical Problem Solving With the MATH Dataset*. arXiv: 2103.03874 [cs.LG].